AWS IAM & IAM Identity Center – Implementation

1. Scope & Audience

Audience

Cloud Engineers, DevOps Engineers, Security Engineers, and AWS Administrators.

Scope

This guide covers **secure human access** to AWS accounts (Console + CLI) using:

- **IAM Identity Center (Recommended)** → Workforce SSO across multiple AWS accounts
- **IAM (Direct)** → Service roles, break-glass access, and limited edge cases

2. Key Concepts (Simplified Mapping)

- **Identity Provider (IdP)**
  The system where users authenticate (password, MFA, conditional access).
  Examples: Azure AD, Okta, or built-in Identity Center directory
- **IAM Identity Center**
  Central service to manage **user access across multiple AWS accounts**
- **Permission Set**
  A reusable permission template that includes:
  - AWS managed policies
  - Optional inline policies
  - Session duration
- When assigned, it automatically creates an **IAM Role** in the target account
- **IAM Role**
  The actual identity used inside AWS accounts to access services
- **IAM (Direct)**
  Traditional IAM (users/roles) mainly used for:
  - Workloads (EC2, Lambda, ECS)
  - Cross-account access
  - Emergency (break-glass) scenarios

3. Implementation: IAM Identity Center

Step 1 — Set up AWS Organizations (Multi-Account Strategy)

- Navigate to AWS Organizations (Management Account)
- Create or verify your Organization
- Create core accounts:
  - Dev
  - Test
  - Prod

Output:
Isolated AWS accounts with better security and access control

Step 2 — Enable IAM Identity Center

- Open IAM Identity Center in your preferred Region
- Click **Enable**
- Ensure integration with AWS Organizations

Output:
Centralized access management is enabled

Step 3 — Configure Identity Source

Option A: Built-in Directory

- Use Identity Center as the identity source
- Create users and groups directly

Option B: External IdP

- Choose SAML / OIDC integration
- Exchange metadata between IdP and AWS
- Validate authentication flow

Output:
Users authenticate via a centralized identity system

Step 4 — Create Users and Groups

Create groups based on job roles:

- AWS-Admins
- AWS-Developers
- AWS-ReadOnly
- AWS-Billing (optional)

Output:
Scalable, group-based access control

Step 5 — Design Permission Sets (RBAC Model)

Create standardized permission sets:

Role	Permissions
Admin	**AdministratorAccess**
PowerUser	**PowerUserAccess (+ restrictions if needed)**
ReadOnly	**ReadOnlyAccess**
Billing	**Billing-specific policies**

Best Practices

- Keep permission sets **modular and reusable**
- Prefer **AWS managed policies**
- Use inline policies only when required
- Set session duration (1–8 hours recommended)

Output:
Reusable and consistent permission structures

Step 6 — Assign Access to Accounts

- Go to **IAM Identity Center → AWS Accounts**
- Select account (Dev/Test/Prod)
- Click **Assign users or groups**
- Attach:
  - Group

- Permission Set

Example Model

Group	Permission	Account Scope
AWS-Admins	**Admin**	**All accounts**
AWS-Developers	**PowerUser**	**Dev only**
AWS-ReadOnly	**ReadOnly**	**Prod / All**

Output:
Users see assigned accounts in AWS access portal

Step 7 — Understand Auto-Created Roles

When a permission set is assigned, AWS creates roles like:

AWSReservedSSO_<PermissionSetName>_<random>

Key Notes:

- Used internally for access
- Managed via Identity Center (not IAM directly)
- Avoid manual modification

Output:
Each account contains SSO-linked IAM roles

Step 8 — User Access Flow

Console Access

1. Open AWS Access Portal
2. Authenticate via IdP
3. Select account and role
4. Access AWS Console

CLI Access (Recommended)

- Use AWS CLI v2 with SSO profiles
- Authenticate once → temporary credentials generated

Output:
Secure access without long-term credentials

4. Direct IAM (Traditional Approach)

When to Use

- Workload identities (EC2, Lambda, ECS)
- Cross-account role access
- Emergency (break-glass) admin access
- Legacy integrations

Avoid for Human Users

- Long-term access keys
- IAM users per account
- Manual permission handling

Minimal Setup (If Required)

1. Create IAM Role (preferred) or IAM User
2. Attach least-privilege policy
3. Enable MFA
4. Rotate access keys regularly

5. Best Practices (Production Ready)

Security

- Follow **least privilege principle**
- Enforce MFA (IdP + root account)
- Eliminate long-term access keys for humans
- Use permission boundaries where needed

Governance

- Use AWS Organizations **Service Control Policies (SCPs)**
- Standardize naming conventions
- Define access approval workflows

Observability & Audit

- Enable **CloudTrail** in all accounts (central logging)
- Use **AWS Config** for compliance monitoring
- Conduct periodic access reviews (quarterly recommended)

6. Glossary

Term	Meaning
AuthN	**Authentication (who you are)**
AuthZ	**Authorization (what you can do)**
SSO	**Single Sign-On**
SCP	**Service Control Policy**

User & Identity Layer
User / Engineer
Login + MFA
Identity Provider\n(Azure AD / Okta)
SAML / OIDC
AWS SSO Layer
IAM Identity Center\n(SSO)
STS Assume Role
Permission Set: ReadOnly
STS Assume Role
STS Assume Role
Permission Set: Admin
Permission Set: PowerUser
AWS Organization
Security & Logs
IAM Role\nAudit / Security
Prod Account
IAM Role\nAWSReservedSSO_ReadOnly
AWS Organizations
Test Account
IAM Role\nAWSReservedSSO_Admin
Dev Account
IAM Role\nAWSReservedSSO_Admin
AWS Services
EC2 | S3 | RDS | Lambda